

Design Decisions and Rationale

A complete record of every significant choice made in producing *Quantifying the Cost of Content-Blindness in LLM Inference Scheduling*, including the alternatives rejected, what each choice buys, and where it is fatally weak.

Written so that any claim in the paper can be traced back to a decision, and so that the weaknesses are stated here as plainly as they are in the paper. If you are asked "why did you do it that way?" in an interview or a viva, the answer is in this file.

Status: submission build complete 2026-08-03 for the ICCCIoT-2026 deadline of 2026-08-04.

1. Venue and track

Decision. ICCCIoT-2026 (IC3IoT-2026), Sri Sairam Engineering College, Chennai. Track 4, Blockchain / Cloud Computing / Big Data Analytics.

Alternatives. Tracks 1 (Signal and Image Processing), 2 (Networks), 3 (IoT, Robotics, Control). Track 5 (Antenna and Microwave) was never CS-relevant.

Why Track 4. The track title contains "Cloud Computing", and the work is a cloud-infrastructure study over 44.1M production requests, so both halves of "cloud" and "big data" are satisfied directly. Within the track's bullet list it maps onto three topics at once: Big Data Architectures (a serving architecture under load), Big Data Models and Algorithms (the scheduling policy and predictor), and Big Data Protection/Privacy (the scheduler is forbidden to read user prompt content).

What it overcomes. Reviewer-assignment risk. A paper that fits only one bullet can land with a reviewer who considers it out of scope.

Fatal weakness. The track's bullet list leans blockchain and biomedical analytics. This paper sits in the "cloud computing + big data architectures" corner of it rather than on any single bullet. If the track's reviewer pool is predominantly blockchain researchers, the systems content may be judged by someone outside its area.

2. Topic selection

Decision. Content-blind admission scheduling for LLM inference, measured on Azure production traces.

Alternatives considered and rejected.

Candidate	Why rejected
Carbon-aware cloud workload scheduling	Good story for German admissions, but requires a carbon-intensity feed joined to cluster traces; two data sources, more integration risk under a 48-hour deadline.
VM rightsizing / overprovisioning waste	Killed on data volume: the Azure 2017/2019 VM CPU-reading traces are 80–160 GB. Not downloadable or processable in the time available.
Serverless cold-start mitigation	Strong prior work (<i>Serverless in the Wild</i> , ATC’20) covers it closely; differentiating would have taken longer than the deadline allowed.
Predictive Kubernetes autoscaling	Real but crowded; hard to say anything new quickly.
Blockchain audit logging for multi-cloud	Fits the track title most literally, but "put logs on a blockchain" is a cliché unless the mechanism is genuinely novel, which it would not have been.
Straggler mitigation in distributed processing	Most crowded of all; lowest chance of a defensible contribution.

Why this one. Three properties no alternative had together: (i) the data is real production traffic, free, and downloadable immediately; (ii) the research question is falsifiable in both directions, so a negative result is still a result; (iii) LLM serving infrastructure is the most market-relevant area in cloud systems right now, which matters given the paper’s secondary purpose.

What it overcomes. The two standard failure modes of a rushed paper: no accessible data, and a question whose only publishable answer is the favourable one.

Fatal weakness. The topic is fashionable, which means it is crowded. Between starting and submitting, we found three pieces of prior art we had to concede (Section 9). A slower literature review before committing would have found them on day one rather than during adversarial review.

3. Dataset

Decision. Azure LLM Inference Trace 2024 (`AzureLLMInferenceTrace_conv_1week.csv` , 1,135,195,393 bytes; `..._code_1week.csv` , 691,989,454 bytes). CC-BY-4.0 / MIT, no authentication.

Alternatives. Azure VM traces V1/V2 (80–160 GB, rejected on size); Azure Functions 2019 (136 MB, viable but a different problem); Azure Packing 2020 (51 MB); the Azure LMM multimodal trace 2025 and LLM 2023 trace, both of which have documentation in the repository but **no downloadable release asset** — we verified this against the GitHub Releases API rather than assuming.

Why this one. It is the only public trace that records, per request, exactly the three fields the research question needs: arrival timestamp, input context length, and generated token count. Critically, it contains **no prompt text**, which means the trace is itself a content-blind view of the workload. The data format enforces the paper’s constraint rather than the paper having to simulate it.

What it overcomes. Any accusation that "content-blind" is an artificial restriction we imposed to make the problem interesting. We could not have read the prompts if we wanted to.

Fatal weaknesses.

- Two workloads, one provider, one week. Every claim is scoped to that.
- May 2024 traffic. It predates widespread reasoning models and, more seriously, predates the era in which prefix caching is universally deployed (Section 12).
- The trace gives token counts but not model identity, hardware, or cache residency, so all three had to be modelled.

4. Simulation instead of real hardware

Decision. A trace-driven discrete-event simulator of a single continuous-batching replica, written from scratch (`src/simulator.py`).

Alternatives.

- *Run vLLM on a real GPU.* Rejected: no GPU available, no budget, and a single meaningful experiment would exceed the deadline.
- *Use an existing simulator* such as Vidur. Rejected: adds a dependency whose internals we would have to understand well enough to defend, in less time than writing 300 lines ourselves.
- *Analytical queueing model (M/G/1 etc.).* Rejected: cannot represent the KV-cache capacity bound or head-of-line admission, which turn out to be the binding constraints.

Why this one. Full control and full inspectability. When adversarial review asked "what fraction of engine time is prefill?", we could add a counter and answer in ten minutes. With a third-party simulator that is a research project.

What it overcomes. The compute constraint entirely: 1,240 simulations run in 141 seconds on 8 CPU workers.

Fatal weakness — the single largest in the paper. Every absolute number is conditional on three cost constants we chose. Adversarial review priced this at 4–5 points out of 100 and it cannot be bought with CPU time. A reviewer is entitled to discount every headline figure by an unknown factor. The sensitivity sweeps (Section 8) show the *direction* is robust; they cannot show the *magnitude* is.

5. The cost model and its parameters

Decision. Prefill affine in prompt tokens (0.28 ms/token, 8 ms fixed); decode affine in batch size (12 ms base + 0.42 ms/request). Parameters for a 13B-parameter model (40 layers, 40 KV heads, head dim 128, fp16) on an 80 GB accelerator, giving a KV budget of 65,917 tokens.

Why these numbers. They are not arbitrary, though the paper is careful not to overclaim this. They fall out of hardware physics:

- Decode is memory-bandwidth-bound. Reading 26 GB of fp16 weights against ~2 TB/s HBM gives ~12.75 ms per iteration, shared across the batch. Hence `decode_ms_base = 12`.
- Prefill is compute-bound at roughly $2 \cdot N \cdot P$ FLOPs, so ~26 GFLOP/token for a 13B model. An A100 at ~30% MFU gives ~0.28 ms/token. Hence `prefill_ms_per_token = 0.28`.

Alternatives. Measuring on hardware (unavailable); citing another paper’s published coefficients (would still not match our modelled configuration, and would import their assumptions without their context).

What it overcomes. The accusation that the parameters were tuned to produce the conclusion. The context-to-output token ratio that drives the result — 15.5:1 and 110.7:1 — is a property of the trace and involves no cost model at all.

Fatal weakness. The paper does not print the physics derivation above; it only says the model is analytical and sweeps it. A reviewer cannot check the coefficients are physically grounded from the paper alone. This was a page-budget casualty and is the first thing to restore in a longer version.

6. Load calibration

Decision. Measure each window’s saturated throughput by driving the engine with an unbounded backlog, then rescale the arrival timeline so offered load ρ takes the values {0.60, 0.75, 0.85, 0.92, 0.98}.

Alternatives. Multiply the raw trace arrival rate by a constant (the first approach tried, and wrong: the trace is fleet-aggregate at 75–87 req/s, orders of magnitude above single-replica capacity of ~1.4–1.8 req/s); or compute capacity analytically (understates it, because the per-iteration base cost amortises over a batch whose size is itself load-dependent).

What it overcomes. It makes load an interpretable control variable and preserves both the arrival-process shape and the request mix exactly.

Fatal weakness, disclosed in the paper. ρ is measured at maximum concurrency while the runs operate at smaller batches, so **measured utilisation exceeds substantially** — 0.91 at $\rho = 0.60$ on conversation. ρ must be read as a control variable, not as utilisation. We verified this does not bias the comparison: throughput agrees across all eight policies to within 0.07% at every load.

7. Window selection

Decision. The first 20,000 requests of each of 20 (conversation) and 11 (code) windows, sampled at even intervals across the full week, at least two hours apart, all after the predictor’s training period.

What we did first, and why it was wrong. The original design took the five *busiest* 20-minute windows. Two defects: (i) two pairs were exactly adjacent in time, so they were not five independent epochs; (ii) selecting on busyness invites a cherry-picking objection. It also turned out to be **unnecessary** — because arrivals are rescaled to a target, a window’s absolute arrival rate is normalised away entirely, so window choice affects only the request mix.

What it overcomes. Covers diurnal and weekday/weekend variation, quadruples the sample, and removes the cherry-picking objection at zero cost.

Fatal weakness, disclosed. We require 20,000 requests per window so all runs are the same length. That is *implicitly* a traffic threshold: it excludes no conversation candidates but **10 of 24 code candidates, which are the quietest periods of that trace**. So "we do not select busy windows" is true of the ranking but not entirely of the filter, and the paper says so.

8. Robustness sweeps

Decision. Sweep (a) output length $\times 1$ to $\times 16$, and (b) prefix-cache hit rate 0% to 90%.

What we did first, and why it was wrong. The original sweep varied the *decode cost coefficient* over a $16\times$ range and reported the result as invariant. Adversarial review showed this proved nothing: the binding constraint is **KV-cache occupancy**, whose per-request footprint is `ctx + gen`, and scaling a cost coefficient leaves that untouched. We had held the mechanism fixed and varied something else, then presented the invariance as robustness.

Why output length. It moves the decode share of work *and* the KV footprint together, and it is the practically important direction: reasoning and agentic workloads generate far more tokens per request than these 2024 traces.

What it overcomes. It shows the improvement over FCFS survives — and on conversation *rises*, 46.0% to 75.1% — even when prefill falls to 11% of engine time. The conclusion therefore does not depend on prefill dominance, which is a stronger claim than the one originally made.

Fatal weaknesses.

- The sweeps use only the **first 8 windows** for compute reasons, so their baseline differs from the main tables (46.0% vs 56.5% at the same load). This is stated in the paper.
 - Sequence length is clamped so no request exceeds 95% of the KV budget, which affects 0.003% of requests at $\times 16$.
 - The two workloads **disagree in direction** under output-length scaling (conversation rises, code falls 63.3% $\rightarrow$ 50.5%). An earlier draft reported only the workload that rose, because the table containing both had been cut for space. Both are now in the prose.
-

9. Baselines, controls and oracles

Decision. Eight policies: FCFS, SJF-Ctx, SJF-Pred, CB-SJF-Work (proposed), LJF-Work and Random (controls), Oracle-SJF and Oracle-Work (upper bounds).

Why each exists.

- **FCFS** is the engine default in vLLM, so it is the real-world baseline.
- **SJF-Ctx** orders by context length alone. It exists to answer "does the learned predictor do anything?" — and the answer is essentially no.

- **SJF-Pred** orders by predicted output length. This is the objective the length-prediction literature pursues, realised under our constraint.
- **Oracle-SJF** / **Oracle-Work** bound what any admission-order policy could achieve with perfect information.
- **LJF-Work** and **Random** were added late, in response to review. LJF-Work is the CB-SJF-Work key *negated*, so it is content-blind in exactly the same way but longest-first; Random orders by an uninformative key.

What the controls overcome — and a claim they corrected. Before they existed, the paper asserted the tail penalty was "a property of shortest-first admission." The controls falsified this. Random reordering, which is not shortest-first at all, damages the tail *more* than we do ($3.25\times$ vs $1.00\times$ on conversation). Since FCFS and Random differ only in whether the ordering key is monotone in arrival time, this isolates **starvation** as the mechanism: the penalty is the price of departing from arrival order at all, and FCFS's strong tail comes from its freedom from starvation rather than from anything it knows.

Fatal weaknesses.

- LJF's magnitude is amplified by head-of-line admission, which fit-tests the largest pending request first. Disclosed in the paper.
 - "Shortest-first is the least harmful" is scoped to the three reorderings we test. Aging or deadline-augmented policies were not tested and might beat it.
 - No preemptive or aging baseline exists. This is the second-largest gap in the paper (priced at 2–3 points), and it is why the paper's own recommendation is *do not deploy any of these without an anti-starvation mechanism*.
-

10. The predictor

Decision. `HistGradientBoostingRegressor` on $\log_{1p}(\text{output length})$, over ten content-blind features, trained on a contiguous 1.5M-request prefix of the first 24 hours, applied unchanged to later windows.

Alternatives. A fine-tuned language model on the prompt (impossible — no prompt text, and it would violate the paper's premise); a simple linear model (rejected — cannot capture the non-monotone context/output relationship); a classifier over quantile buckets (rejected — regression preserves ordering, which is what scheduling needs).

Two bugs found and fixed.

1. *Label leakage risk.* All rolling statistics are shifted by one request so no request can see its own or any future outcome, and `features.assert_content_blind()` fails the run if any feature exceeds 0.95 rank correlation with the target.
2. *Train/serve skew.* The training set was originally a **strided** subsample, which stretched inter-arrival times and widened rolling windows at training time only — five of ten features were on a different scale at train and serve. Fixed to a contiguous prefix. Notably this made the negative result *harder* to obtain, which is the right direction.

Fatal weakness, and it is the paper's own finding. The predictor is not worth deploying. It contributes between 0.11% and +3.45% over ordering by raw context length, is negative in 5 of 10 cells, and reaches $p < 0.05$ only at the highest load. The paper says so explicitly and recommends against it. It becomes worth 21.1% only when generations are $16\times$ longer than in these traces.

11. Metrics

Decision. Mean normalised latency (end-to-end latency per output token) as primary; 99th-percentile end-to-end latency; SLO attainment against a 2 s TTFT and 100 ms TBT target.

Why normalised latency. It is the standard metric in the continuous-batching literature and it prevents long requests from dominating purely by being long.

Fatal weakness, tested rather than assumed. For a one-token request, normalised latency degenerates to raw latency, so short requests can dominate the mean in a way that rewards shortest-first ordering **by construction**. They do: at $\alpha=0.92$, requests emitting 4 tokens are 10.9% and 36.5% of arrivals but 50.2% and 78.6% of total normalised latency. We therefore re-ran the headline restricted to requests emitting 8 tokens: the improvement is 47.04% and 48.5% against 51.5% and 58.8%, positive in all 31 windows. Inflated, but not an artifact. The inflation grows as load falls, and the paper reports the unfavourable end (11.1% vs 22.9% on code at $\alpha=0.75$).

A related degeneracy: time-between-tokens is 0 for a one-token request, so a fraction of code requests pass the TBT target unconditionally.

12. Prefix caching — the threat we cannot answer

Decision. Model it as a reduction in *executed* prefill while the content-blind scheduler still observes the *full* context length, because cache-hit length depends on token identity and computing it would require reading content.

Why this matters most. Production stacks run automatic prefix caching, and multi-turn conversation — the headline workload — has the highest hit rates, because each turn re-sends the history. The traces record context length but not cache residency, so this cannot be measured, only modelled.

Result. At a 90% hit rate the improvement over FCFS falls from 46.0% to 12.8% (conversation) and 63.3% to 14.8% (code). On code the decline is a cliff, not a slope: 48.6% at 75% versus 14.8% at 90%.

A correction we had to make. The first version of this sweep denied cache residency to the *oracle* as well, so both policies converged because both were handicapped — and the paper wrongly read the resulting high "recovered gap" as evidence that caching penalises all schedulers equally. Against a properly cache-aware oracle the recovered gap falls from 85.5% to **66.0%** (conversation) and 74.5% to **69.8%** (code).

Fatal weakness. This is the paper's largest live threat and it is disclosed as the main open problem rather than resolved.

13. Statistics

Decision. Paired one-sided Wilcoxon signed-rank tests across windows, with per-window ranges reported alongside aggregate figures.

Why paired. The same windows are replayed under every policy, so pairing is valid and far more powerful than an unpaired test at these sample sizes.

Three problems found and fixed.

1. *n=5 floor.* The original five windows made $p=0.031$ the smallest attainable value, so every cell hit the floor and the p-value carried one bit of information. Fixed by raising n to 20/11.
2. *Ratio of means vs mean of ratios.* The headline improvement is a ratio of means, which is weighted by whichever window has the worst FCFS run. The paper now reports both (51.5% vs 47.0% on conversation) and the per-window range (6% to 77%).
3. *Estimator mismatch, caught last.* Table II used mean-of-ratios for the P99 row while the text used ratio-of-means, so the same cell printed 1.70 and 2.05. Worse, the comparison against Oracle-Work **reversed** once made consistent: the oracle's tail is marginally *better* than ours, not worse. Corrected.

Fatal weakness. Cohen's d was originally quoted as a range; it was dropped because across deterministic replays it measures between-window heterogeneity, not sampling error, and it was anti-correlated with practical significance ($d=3.91$ for a 1.07% effect, $d=0.78$ for the 58.3% headline).

14. Reproducibility infrastructure

Decision. Every number in the paper is generated programmatically and verified by `src/audit_numbers.py`, which pairs each printed value with the file, column and selection that must produce it. 167 checks, plus a

completeness pass that regexes every numeral out of `main.tex` and reports anything uncovered.

Why it exists. Not as decoration. During revision, one number — SLO attainment at low load — survived a rewrite without being regenerated, printing 39.2%/17.6% when the regenerated results gave 54.3%/34.5%. That single stale value falsified the paper’s own printed claim that no number is transcribed by hand. The audit exists so that claim is checkable rather than asserted, and it caught two further errors while being written.

What it overcomes. The most common quiet defect in empirical papers: the abstract disagreeing with the table because someone edited one and not the other.

Fatal weaknesses, both real.

- It verifies **transcription, not derivation**. It proves the LaTeX matches `results/`; it does not prove `results/` matches the experiment. If the gen8 filter were implemented wrongly, the audit would still be green.
- Completeness is **by numeral value, not by claim**. A claim whose number happens to coincide with another checked value is reported as covered. This bit us: the "within 0.06%" throughput claim was certified by collision with an unrelated p-value, and the claim was wrong (the true spread is 0.063%).

15. Tooling

Choice	Alternatives	Why	Weakness
LaTeX / IEEEtran	Word + python-docx	Most reliable route to a compliant IEEE PDF; MiKTeX 25.12 verified working before any content was written	Page-count control is fiddly; layout reflow absorbs small cuts unpredictably
Python 3.14	Older Python	Already installed	Very new; scientific wheels were a real risk, so availability was verified before committing (all resolved to prebuilt cp314 wheels)
matplotlib, <code>pdf.fonttype=42</code>	Default Type 3 output	IEEE PDF eXpress rejects Type 3 fonts , and the failure only surfaces at upload time	None; this is a strict improvement
Okabe-Ito palette + distinct markers and linestyles	Colour alone	IEEE prints greyscale, so identity must never be colour-alone; palette was validated with a CVD checker rather than eyeballed	Six series in one figure is busy
Parquet cache, categorical workload column	Re-parsing CSV	27M Python strings for the workload column alone cost over a gigabyte of RSS and broke the process pool	None

16. Review history

Five rounds of adversarial review, scored against the reviewer’s own rubric: **54 → 66 → 74 → 78 → 80**, against a stated structural ceiling of 82.

What each round caught, in order of severity:

1. **A suppressed negative result.** CB-SJF-Work has a worse P99 than FCFS in 31 of 33 code cells. The paper reported the mean-latency win from =0.98 and the tail only from =0.92 — different operating

points. Now has its own subsection and an abstract sentence.

2. **A stale number** that falsified the reproducibility claim (Section 14).
3. **A fix that deleted a counterexample.** Cutting a table for space removed the code workload’s trajectory, which falls where conversation’s rises.
4. **An untested claim in the abstract.** "Worse than FCFS on code" was asserted with no test while the predictor’s own gain was held to $p < 0.01$. Tested: 8/11 windows, $p = 0.06$. Now stated with that precision.
5. **"Up to" applied to a mean.** The worst-window P99 ratios are $4.57\times$ and $6.06\times$, not the $1.9\times/3.1\times$ means.
6. **Three false universals** in captions and contributions, each falsified by the table it described.

Why this history is worth keeping. Every one of these was found before a reviewer saw the paper, and roughly half were introduced *by* an earlier fix. That is the strongest argument for adversarial review as a process: the failure mode is not carelessness, it is that each repair perturbs something else.

17. The honest ceiling

Three gaps cap this work, and none is closable by more effort on the same substrate:

1. **No real-hardware validation** (worth $\sim 4\text{--}5$ points). Every absolute number is conditional on modelled constants.
2. **No preemption or aging** ($\sim 2\text{--}3$ points). The paper’s own advice is not to deploy its policy without one.
3. **The policy is not novel** ($\sim 3\text{--}4$ points). It is SJF-Ctx with an extra term that earns nothing, and the paper says so in Section IV-B. FastServe and PecSched already schedule on input length.

The contribution is therefore a **measurement**, not a mechanism: nobody had quantified what content-blindness costs an admission scheduler, and 90.3% / 79.3% is a number that did not previously exist.

The natural next paper is any one of: running the top three policies on one real GPU under vLLM against a replayed window to test whether the ordering conclusions hold; adding aging or preemption to remove the tail regression; or extending to prefix-cache-aware scheduling, which is the open problem this work exposes.